Lesson 14 In-Class Worksheet (SOLUTION KEY)
Reinforcement Learning II: Policy Iteration

Part 1: Instructor-Led Walkthrough

Objective: Apply Policy Iteration to a simple decision problem.
Scenario: An autonomous sentry is stationed at a base (S). It must decide between two actions: *Patrol* or *Recharge*.

- **Action: Patrol (P)** $\rightarrow$ Immediate Reward: $+5$
- **Action: Recharge (R)** $\rightarrow$ Immediate Reward: 0

Assume the discount factor $\gamma = 0.9$ and both actions loop right back to the base state (S) deterministically.

Step 1: Initialization
We initialize the sentry with a flawed, random policy. Let the initial playbook be:
$\pi_0(S) =$ Recharge.

Step 2: Policy Evaluation
Calculate the value of the state under this specific playbook (π_0).
Hint: $V^{\pi}(s) = R(s, \pi(s), s') + \gamma V^{\pi}(s')$
Because the sentry only recharges, it gains 0 reward each turn:
$V^{\pi_0}(S) = 0 + 0.9 \cdot V^{\pi_0}(S) \implies V^{\pi_0}(S) = 0$

Step 3: Policy Extraction (Improvement)
Run a one-step lookahead against the environment to find the best immediate action based on V^{π_0}.
Expected utility of Patrol:
$R(S, P, S) + \gamma V^{\pi_0}(S) = 5 + 0.9(0) = 5$
Expected utility of Recharge:
$R(S, R, S) + \gamma V^{\pi_0}(S) = 0 + 0.9(0) = 0$
New extracted policy $\pi_1(S) =$ **Patrol** (since $5 > 0$)

Step 4: Convergence Verification
Evaluate $\pi_1(S)$ to find $V^{\pi_1}(S)$, then run extraction again to find $\pi_2(S)$.
Does $\pi_2 = \pi_1$?
Evaluation of π_1:
$V^{\pi_1}(S) = 5 + 0.9 \cdot V^{\pi_1}(S) \implies 0.1V^{\pi_1}(S) = 5 \implies V^{\pi_1}(S) = 50.$
Extraction for π_2:

Utility of Patrol $= 5 + 0.9(50) = 50$.
Utility of Recharge $= 0 + 0.9(50) = 45$.
Since $50 > 45$, $\pi_2(S) =$ Patrol.
Yes, π_2 is perfectly identical to π_1, so the algorithm halts. The optimal policy is to always Patrol.

Part 2: Student On Their Own

Instructions: Complete the following questions to check your understanding of the lesson objectives.

Question 1: Value Iteration vs. Policy Iteration
While both algorithms solve Markov Decision Processes, they take different operational paths. In your own words, explain the fundamental difference between how Value Iteration and Policy Iteration reach the optimal solution. Which one focuses on the "playbook" at every step?
Answer: Value Iteration focuses entirely on calculating raw numerical values until they mathematically converge, extracting the actual policy only once at the very end. Policy Iteration focuses heavily on the "playbook" by continuously extracting and evaluating intermediate policies. Policy iteration usually converges in far fewer overall steps, even though the math inside each step is heavier.

Question 2: Online vs. Offline Planning
Solving these models with Policy Iteration is considered *Offline Planning*. Explain the key difference between Offline Planning and *Reinforcement Learning* (Online Planning). How does the agent's interaction with the environment differ?
Answer: In Offline Planning, the AI determines all quantities purely through computation because it has perfect knowledge of the environmental details (T and R). It does not take physical actions. In Reinforcement Learning, the environment is unknown, so the agent is placed directly into it. The agent must physically learn via trial and error based strictly on the raw rewards it observes from its interactions (samples of outcomes).

Question 3: Algorithm Exits
In Policy Iteration, how does the AI agent definitively know when to stop running its loops and exit the algorithm?
Answer: The algorithm knows it has found the optimal solution when the newly extracted policy is mathematically identical to the old one ($\pi_{i+1} = \pi_i$). If the playbook doesn't change after an extraction step, it halts.

Question 4: Policy Evaluation Math
An AI commander must calculate the exact expected utility of following an established Standard Operating Procedure (SOP). To do this, the commander uses Policy Evaluation. Why is the $\max_a$ operator removed from the Bellman equation during this specific calculation?
Answer: The $\max_a$ operator represents the process of searching for the best possible action. In Policy Evaluation, the actions are already pre-decided by the fixed SOP (the chosen policy). Since the agent isn't choosing an action but rather just following instructions, there is no need to search or maximize over actions, turning the search into a straightforward calculation.